

ESTUDIO — Cómo interactúan las capas del sistema

Material de estudio para el final del TFC Bajoneá. Explicación transversal de **por qué** el código está separado en capas, qué hace cada una, y cómo viaja un dato desde que el usuario lo escribe hasta que se guarda en la base y vuelve.

Parte 1 — La idea, en una frase

El código está separado en capas para que cada una tenga una sola responsabilidad, y para que un cambio en una no obligue a tocar las demás.

Todo lo que sigue es desarrollar esa frase.

Parte 2 — El problema que resuelven las capas

Cómo sería sin capas

Imaginate escribir todo junto: un solo método que recibe el pedido HTTP, valida, arma el SQL, consulta la base, decide la lógica de negocio y arma el JSON de respuesta.

Al principio funciona. El problema aparece después:

Querés hacer...	Y te encontrás con que...
Cambiar MySQL por PostgreSQL	Tenés SQL desparramado en 50 métodos.
Exponer lo mismo por otro medio	La lógica está pegada al código HTTP.
Probar una regla de negocio	Necesitás levantar un servidor y una base para probar un <code>if</code> .
Cambiar el formato de las respuestas	Está armado a mano en 60 lugares distintos.
Entender por qué no deja hacer algo	Tenés que leer 300 líneas mezcladas.

Cómo es con capas

Cada capa hace **una sola cosa** y habla **solo con la de al lado**:

CONTROLLER	"recibo HTTP y devuelvo HTTP"
SERVICE	"decido, valido y coordino"
REPOSITORY	"traigo y guardo datos"
BASE DE DATOS	

...y atravesando todo:

- DTO — el formato en que viajan los datos
- ENTITY — el formato en que se guardan
- VALIDATION — las reglas de formato de lo que entra
- EXCEPTIONS — cómo se comunican los errores
- CONFIG — los preparativos del arranque

La consecuencia práctica: cambiar de base de datos toca solo los repositories. Cambiar el formato de respuesta toca solo los controllers y el `ApiResponse` . Cambiar una regla de negocio toca solo un service.

Parte 3 — Qué hace exactamente cada capa

config/ — los preparativos

Responsabilidad: todo lo que Spring hace **una sola vez, al arrancar**, para que el resto funcione.

Qué configura	Archivos
La seguridad completa (JWT, permisos por rol, filtros)	<code>config/security/</code> — 7 clases
El cliente de Cloudinary	<code>CloudinaryConfig</code>
El cliente de Resend	<code>MailConfig</code>
La documentación Swagger	<code>OpenApiConfig</code>

Lo que NO hace: ninguna regla de negocio.

Cómo se conecta: los `@Bean` que crea (como `PasswordEncoder` o `Cloudinary`) los inyecta Spring en los services que los piden.

controllers/ — la puerta de entrada

Responsabilidad: traducir entre HTTP y Java. Tres pasos y nada más:

1. Recibir el pedido y convertir el JSON en un DTO.
2. Llamar al service.
3. Devolver la respuesta con el código HTTP correcto.

```
@PostMapping
public ResponseEntity<ApiResponse<ProductoResponseDTO>> crearProducto(
    @Valid @RequestBody ProductoRequestDTO request,
    @AuthenticationPrincipal AuthenticatedUser usuario) {
    ProductoResponseDTO response = productoService.crearProducto(usuario.userId(), request);
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(new ApiResponse<>("Producto creado correctamente", response));
}
```

Tres líneas. Cero lógica.

Lo que NO hace:

- No valida reglas de negocio (`@Valid` valida formato, no negocio).
- No consulta la base.
- **Nunca devuelve una entidad JPA** — es regla del proyecto.
- No arma respuestas de error (las arma el `GlobalExceptionHandler`).

Cómo se conecta: recibe DTOs de request, llama a services, devuelve DTOs de response envueltos en `ApiResponse` .

services/ — donde se piensa

Responsabilidad: toda la lógica de negocio.

Cuatro cosas:

1. **Validar reglas de negocio** — "no podés confirmar un pedido con el carrito vacío".
2. **Orquestar repositories** — llamar a varios para armar una operación.
3. **Convertir entidades en DTOs** — el mapeo, hecho a mano.
4. **Coordinar con otros services** — `PedidoService` le pide a `NotificacionService` que avise.

Lo que NO hace:

- No sabe nada de HTTP (no ve status codes ni headers).
- No escribe SQL.

Cómo se conecta: recibe DTOs del controller y el `userId` del token; usa repositories y otros services; devuelve DTOs; lanza excepciones cuando algo no se puede.

Detalle que muestra la separación: los services **no devuelven códigos HTTP**, lanzan excepciones. Traducir esa excepción a un 409 es trabajo del handler. Por eso el service podría usarse desde un contexto que no sea HTTP sin cambiar nada.

repositories/ — el acceso a datos

Responsabilidad: traer y guardar. Nada más.

```
public interface ProductoRepository extends JpaRepository<Producto, Integer> {
    List<Producto> findByComercioId(Integer comercioId);
    long countByCategoriaId(Integer categoriaId);
}
```

Casi no tienen código: son **interfaces**, y Spring Data genera la implementación en tiempo de ejecución leyendo el nombre de los métodos.

Lo que NO hacen: ninguna decisión. Si hay que comparar, calcular o validar, eso es del service. Es la regla de que "los repositories son tontos".

Cómo se conectan: los usan los services; devuelven entidades (nunca DTOs).

entities/ — la forma de los datos guardados

Responsabilidad: ser el espejo de una tabla en Java.

```
@Entity
@Table(name = "producto")
public class Producto {
    @Id @GeneratedValue(strategy = IDENTITY)
    private Integer id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "comercio_id", nullable = false)
    private Comercio comercio;
    ...
}
```

Lo que NO hacen: ninguna lógica, ninguna validación, y ni un solo comentario (regla del proyecto — toda explicación vive en la documentación).

Cómo se conectan: las manejan los repositories; los services las convierten a DTOs. **Nunca salen a la respuesta HTTP.**

dto/ — la forma de los datos que viajan

Responsabilidad: transportar datos entre el mundo exterior y el sistema.

- **Request:** lo que entra. Llevan las validaciones.
- **Response:** lo que sale. Son **inmutables** (todos los campos `final`).

Por qué existen aunque parezcan "una entidad repetida": las 5 razones del glosario, y la mejor es la de los tres DTOs de comercio: el público no lleva el DNI del representante, el propio sí, el del admin lleva más todavía. **Con una sola entidad no podrías hacer esa distinción.**

validation/ — las reglas de formato

Responsabilidad: validar el **formato** de lo que entra, con anotaciones.

12 anotaciones custom (`@ValidarCuit` , `@ValidarTelefonoArgentino` ...) más las estándar.

Se aplican SOLO en los DTOs de request, nunca en las entities. El motivo está escrito en el `package-info.java` de la carpeta: "*las entities representan el dato ya persistido y válido; la validación de entrada ocurre en el borde HTTP*".

Sus límites: valida un campo a la vez y no puede consultar la base. Por eso hay reglas que necesariamente viven en el service.

exceptions/ — cómo se comunican los errores

Responsabilidad: dar un lenguaje común de errores entre capas.

4 excepciones propias + 1 handler:

Excepción	Status	Cuándo
RecursoNoEncontradoException	404	No existe.
ConflictoDeNegocioException	409	Choca con una regla o con el estado actual.
ValidacionException	400	Regla de negocio que no cabe en una anotación.
CredencialesInvalidasException	401	Credenciales o token inválidos.

Cómo se conecta: el service lanza y se olvida; el `GlobalExceptionHandler` traduce.

Esto es lo que le permite al service no saber nada de HTTP.

enums/ y util/ — el apoyo

enums/ : 24 tipos de lista cerrada. Los usan entities, DTOs y services por igual.

util/ : 2 clases con funciones puras que varios services necesitan — `TextUtils` (Title Case, normalización de URL y código postal) y `ComercioValidaciones` (la regla de al menos una modalidad de entrega).

Parte 4 — El viaje completo de un dato

Vamos a seguir un caso concreto de punta a punta: **Marcelo crea un producto.**

Ida — del navegador a la base

Paso 0 — El navegador (antes de salir)

Marcelo escribe "pizza muzzarella", precio 8500, categoría "Pizzas".

`validators.js` valida en el momento: nombre válido, precio positivo. **Es UX, no seguridad** — se puede saltar desde las herramientas de desarrollo.

`api.js` arma el request:

```
apiFetch('/productos', {
  method: 'POST',
  body: { nombre: 'pizza muzzarella', precio: 8500, categoriaId: 3, tagIds: [1, 4] }
});
```

Le agrega solo el header `Authorization: Bearer <token>` .

Paso 1 — Los filtros de seguridad

1. RateLimitFotoRegistroFilter

→ no es ruta de firma, pasa de largo
2. JwtAuthenticationFilter

→ valida la firma del token

→ consulta Sesion.activa en la BASE

→ arma AuthenticatedUser(userId=7, rol=DUENO, ...)

→ lo carga en el SecurityContextHolder
3. Reglas de SecurityConfig

→ `/api/v1/productos/**` requiere DUENO ✓

Si algo falla acá, el pedido NUNCA llega al controller: 401 lo escribe `CustomAuthenticationEntryPoint` , 403 lo escribe `CustomAccessDeniedHandler` .

Paso 2 — Spring convierte el JSON en DTO

Jackson lee el JSON y llama a los setters del `ProductoRequestDTO` .

En este DTO no hay setters manuales, pero en `RegistroClienteRequestDTO` sí — y ahí es donde se normaliza (trim, minúsculas, sacar puntos del DNI) antes de que corra la validación.

Paso 3 — @Valid dispara Bean Validation

- @NotBlank + @Pattern + @Size(max=150)

sobre nombre
- @NotNull + @Positive + @Digits

sobre precio
- @NotNull

sobre categoriaId
- @Size(max=5)

sobre tagIds

Si algo falla: Spring lanza `MethodArgumentNotValidException` antes de entrar al método, y el handler devuelve 400 con un mapa `{campo: mensaje}` .

El dato nunca llega al service si el formato está mal.

Paso 4 — El controller

```
ProductoResponseDTO response = productoService.crearProducto(usuario.userId(), request);
```

Fijate: le pasa el `userId` sacado del token, no del request. Marcelo no puede decir "creá este producto en el comercio 9".

Paso 5 — El service piensa

`@Transactional` abre la transacción.

1. Resuelve el comercio: `comercioRepository.findByDuenoId(7)` → si no hay, 404.
2. Busca la categoría: `categoriaRepository.findById(3)` → si no existe, 404.
3. Normaliza: `TextoUtils.aTitleCase("pizza muzzarella")` → "Pizza Muzzarella" .
4. Construye la entidad con el builder, estado `DISPONIBLE` (decisión del service, no del DTO).
5. Guarda: `productoRepository.save(producto)` .
6. Asigna los tags: una fila de `ProductoTag` por cada uno.

Paso 6 — El repository traduce a SQL

```
productoRepository.save(producto)
```

Hibernate genera:

```
INSERT INTO producto (comercio_id, categoria_id, nombre, descripcion, precio, estado, fecha_creacion)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

El service nunca vio ese SQL.

Paso 7 — La base valida lo último

MySQL chequea los `NOT NULL` , las FK y los `UNIQUE` .

Si algo falla → `DataIntegrityViolationException` → el handler distingue: si es un `UNIQUE` (código 1062) devuelve **409**; si es un `NOT NULL` o una FK, devuelve **500** y lo loguea, porque eso es un bug de programación, no culpa del usuario.

Vuelta — de la base al navegador

Paso 8 — El service mapea a DTO

```
private ProductoResponseDTO aResponseDTO(Producto producto) {
    List<ImagenProductoResponseDTO> imagenes = imagenProductoRepository
        .findByProductoIdOrderByOrdenAsc(producto.getId()).stream().map(...).toList();
    List<String> tags = productoTagRepository
        .findByProductoId(producto.getId()).stream().map(pt -> pt.getTag().getNombre()).toList();

    return new ProductoResponseDTO(
        producto.getId(), producto.getNombre(), ...,
        producto.getCategoria().getNombre(), // ← resuelve el nombre
        producto.getComercio().getNombre(),   // ← resuelve el nombre
        producto.getEstado(), imagenes, tags);
}
```

Dos cosas para notar:

- Resuelve los nombres de categoría y comercio, para que el frontend no tenga que hacer llamadas extra.
- Sale un DTO, no la entidad.

Paso 9 — Se cierra la transacción

Si nada falló → **COMMIT**. Si hubo una excepción → **ROLLBACK**: se deshace todo, incluidos los tags que ya se habían guardado.

Paso 10 — El controller envuelve

```
return ResponseEntity.status(HttpStatus.CREATED)
    .body(new ApiResponse<>("Producto creado correctamente", response));
```

201 Created + {mensaje, data} .

Paso 11 — El JSON sale

```
{
  "mensaje": "Producto creado correctamente",
  "data": {
    "id": 42,
    "nombre": "Pizza Muzzarella",
    "precio": 8500,
    "nombreCategoria": "Pizzas",
    "nombreComercio": "Pizzas del Sur",
    "estado": "DISPONIBLE",
    "imagenes": [],
    "tags": ["abundante", "vegetariano"]
  }
}
```

Paso 12 — El navegador lo recibe

`apiFetch` chequea el status, y como es 2xx devuelve directamente el `data` . La pantalla muestra el toast "Producto creado correctamente" y redirige a la galería.

El viaje completo, de un vistazo



Parte 5 — Las cuatro capas de validación

Un mismo dato se valida **cuatro veces**, y cada una tiene un motivo distinto. Es lo que se llama **defensa en profundidad**.

#	Dónde	Qué valida	Se puede saltar
1	Navegador (validators.js)	Formato, para dar feedback inmediato	Sí, con las herramientas de desarrollo
2	DTO (Bean Validation)	Formato de cada campo	No
3	Service	Reglas de negocio y de estado	No
4	Base de datos	UNIQUE , NOT NULL , FK	No

Por qué las cuatro, y no solo la última

El caso del email único lo muestra perfecto:

1. Navegador:	"eso no parece un email"	→ UX inmediata
2. DTO:	@ValidarFormatoEmail	→ formato garantizado
3. Service:	existsByEmail() → 409	→ mensaje claro y específico
4. Base:	UNIQUE en la columna	→ GARANTÍA REAL

¿Por qué el paso 3 si el 4 ya lo garantiza? Porque entre el existsByEmail y el INSERT puede meterse otro request — es una **condición de carrera check-then-act**. El paso 3 no garantiza nada: existe para dar un mensaje claro ("ese email ya está registrado") en el 99,9% de los casos, en vez del mensaje genérico del UNIQUE .

La **única garantía real** es la base. Y por eso el handler de DataIntegrityViolationException existe: para traducir ese choque a un 409 legible en vez de un 500 crudo.

Y por qué el frontend valida si se puede saltar

	Frontend	Backend
Para qué	Experiencia	Seguridad
Cuándo avisa	Mientras escribís	Al mandar
Confiable	No	Sí

El **backend nunca confía en el frontend**. Un atacante puede desactivar JavaScript o mandar el request con curl. Las validaciones del navegador existen para que el usuario legítimo no espere el viaje al servidor por un error de tipeo.

Parte 6 — Dónde va cada tipo de validación

Esta tabla es muy útil porque la pregunta "¿cómo decidís dónde poner una validación?" es de las que más caen:

Tipo de regla	Dónde va	Ejemplo
Obligatoriedad	@NotBlank / @NotNull en el DTO	El email no puede estar vacío
Formato simple	Anotación estándar	@Size(max = 100)
Formato con algoritmo	Anotación custom	@ValidarCuit (dígito verificador)
Dos campos, un solo lugar	El service	direccionId obligatorio si es domicilio
Dos campos, varios lugares	Clase utilitaria	ComercioValidaciones
Requiere consultar la base	El service	Que el email no exista
Depende del estado actual	El service	Solo se acepta un pedido PENDIENTE
Unicidad, garantía final	UNIQUE de la base	El email en usuario

El criterio general: cuanto más específica y "de formato" es la regla, más cerca del borde va. Cuanto más "de negocio" y dependiente del contexto, más adentro.

Parte 7 — Cómo se comunican las capas: los contratos

Qué le da cada capa a la siguiente

```
Navegador → Controller : JSON + header Authorization
Controller → Service   : DTO de request + userId (del TOKEN, no del request)
Service → Repository  : entidades y parámetros de búsqueda
Repository → Service   : entidades (nunca DTOs)
Service → Controller  : DTO de response, o una EXCEPCIÓN
Controller → Navegador : ApiResponse{mensaje, data} + status HTTP
```

Las cuatro reglas que hacen que esto funcione

- El controller nunca devuelve una entidad. Siempre un DTO envuelto en ApiResponse . Es regla del proyecto, verificada con un hook automático.
- El repository nunca devuelve un DTO. Su trabajo es traer entidades. El mapeo es del service.
- El service nunca sabe de HTTP. No ve status codes. Lanza excepciones y el handler las traduce.
- Toda respuesta tiene el mismo formato. {mensaje, data} , sea éxito o error. Así el frontend siempre sabe cómo leerla.

Parte 8 — Los conceptos transversales que atraviesan todas las capas

El userId que baja desde el token

```
JwtAuthenticationFilter → arma AuthenticatedUser(userId, sesionId, email, rol)
Controller              → @AuthenticationPrincipal se lo inyecta
Controller              → se lo pasa al service
Service                 → filtra todas sus consultas por ese userId
```

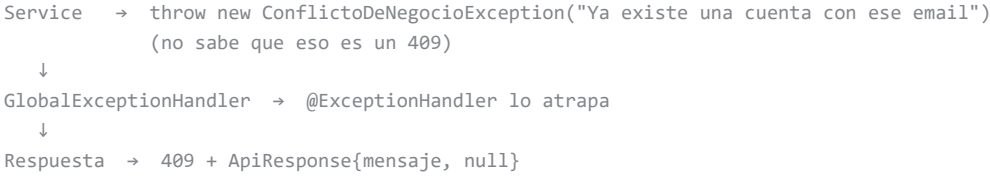
Nunca viene del request. Es lo que hace imposible que alguien vea datos ajenos cambiando un número en la URL.

La transacción que envuelve todo el service

```
@Transactional
├─ el service hace 6 operaciones sobre la base
├─ si todo sale bien           → COMMIT
└─ si sale una excepción      → ROLLBACK (se deshace todo)
```

El caso concreto: confirmarPedido crea el pedido, 3 detalles, vacía el carrito y crea una notificación. Si la operación 4 falla, sin transacción quedaría un pedido a medio armar con el carrito todavía lleno.

Las excepciones que suben desde el service



Esa separación es lo que le permite al service ser independiente de HTTP.

Parte 9 — Cómo contar esto en la mesa

La respuesta corta (30 segundos)

Bajoneá tiene una arquitectura en capas. El **controller** recibe el HTTP y delega — no tiene lógica. El **service** es donde vive toda la lógica de negocio: valida reglas, orquesta y convierte entidades en DTOs. El **repository** solo habla con la base. Y hay tres piezas transversales: los **DTOs**, que son el formato en que viajan los datos y me permiten mostrar lo mismo con distinto detalle según el rol; las **entities**, que son el espejo de las tablas; y el **manejo de excepciones**, que traduce los errores de negocio a códigos HTTP en un solo lugar.

La respuesta con ejemplo (2 minutos)

Te lo cuento con un caso. Un comercio crea un producto.

El request llega con el token en el header. Antes de llegar a cualquier código mío, pasa por dos **filtros de seguridad**: uno valida la firma del JWT y consulta que la sesión siga activa en la base, y después las reglas de `SecurityConfig` chequean que el rol alcance.

Ahí Spring convierte el JSON en un **DTO de request**, y `@Valid` dispara las validaciones de formato: si el precio es negativo, corta ahí con un 400 y **nunca llega al service**.

El **controller** hace tres líneas: llama al service pasándole el `userId` **sacado del token**, no del request. Eso es importante: si viniera del request, cualquiera podría crear productos en el comercio de otro.

El **service** es donde se piensa. Abre una transacción, resuelve cuál es el comercio de ese usuario, verifica que la categoría exista, normaliza el nombre a Title Case y construye la **entity** con estado `DISPONIBLE` — eso lo decide el service, no viene en el DTO.

El **repository** guarda. Yo escribo `save(producto)` y Hibernate genera el `INSERT`. Nunca escribo SQL.

La vuelta es al revés: el service convierte la entity en un **DTO de response** —resolviendo de paso el nombre de la categoría y del comercio, para que el frontend no haga llamadas extra —, se hace commit, y el controller lo envuelve en `ApiResponse` con un 201.

Y si algo falla en el camino, el service **lanza una excepción** sin saber nada de HTTP; una clase con `@RestControllerAdvice` la traduce al código correcto. Eso es lo que mantiene la lógica de negocio independiente del protocolo.

Parte 10 — Preguntas típicas y cómo responderlas

"¿Por qué separaste en capas?" Para que cada parte tenga una sola responsabilidad y un cambio no se propague. Si cambio de MySQL a otra base, toco solo los repositories. Si cambio el formato de respuesta, toco solo los controllers. La lógica de negocio queda aislada de las dos cosas.

"¿Qué pasa si el controller tuviera lógica de negocio?" Quedaría atada al protocolo HTTP. No podría reutilizarla desde otro contexto, y para probar una regla necesitaría levantar un servidor entero en vez de instanciar una clase.

"¿Por qué el service no devuelve códigos HTTP?" Porque el service no sabe que existe HTTP. Lanza una excepción de negocio y el `GlobalExceptionHandler` decide que eso es un 409. Es lo que mantiene la lógica independiente del protocolo.

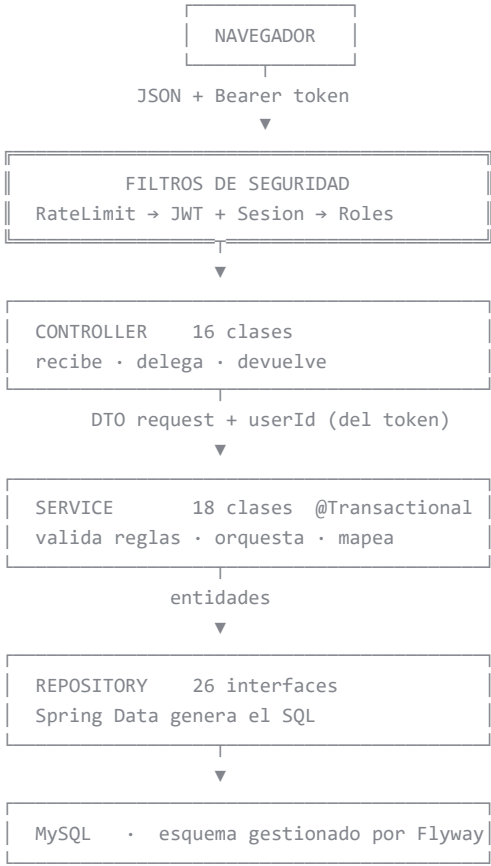
"Contame cómo viaja un dato." *(La respuesta larga de la Parte 9.)*

"¿Por qué validás en cuatro lugares?" Porque cada capa valida algo distinto. El navegador da feedback inmediato pero se puede saltar. El DTO valida formato. El service valida reglas de negocio y estado. Y la base es la única garantía real ante una condición de carrera. Es defensa en profundidad.

"¿Cuál es la capa más importante?" El service, sin dudas. Ahí está todo lo que hace que el sistema sea Bajoneá y no cualquier otra aplicación. El resto es infraestructura: recibir HTTP y hablar con una base lo hace cualquier sistema.

"¿Qué pasa si querés agregar una funcionalidad nueva?" El camino es siempre el mismo: la entity si hace falta una tabla, el repository, el DTO de request y de response, el service con la lógica, y el controller que expone el endpoint. De hecho el proyecto tiene una skill documentada (`generar-capa-crud`) precisamente con ese orden.

Parte 11 — Un diagrama para memorizar



- Atravesando todo:
- dto/ (62) el formato en que viajan los datos
 - entities/ (27) el formato en que se guardan
 - enums/ (24) los valores posibles
 - validation/ (12) las reglas de formato
 - exceptions/ (5) el lenguaje de errores → HTTP
 - util/ (2) funciones compartidas
 - config/ (10) los preparativos del arranque

Índice de lo cubierto en este documento

Parte	Contenido
1	La idea en una frase.
2	El problema que resuelven las capas.
3	Qué hace exactamente cada capa (config, controllers, services, repositories, entities, dto, validation, exceptions, enums, util).
4	El viaje completo de un dato, paso a paso, ida y vuelta.
5	Las cuatro capas de validación y por qué hacen falta las cuatro.
6	Dónde va cada tipo de validación.
7	Los contratos entre capas y las cuatro reglas que los sostienen.
8	Los conceptos transversales: el userId del token, la transacción, las excepciones.
9	Cómo contarlo en la mesa (versión corta y versión con ejemplo).
10	Preguntas típicas con respuesta.
11	El diagrama para memorizar.